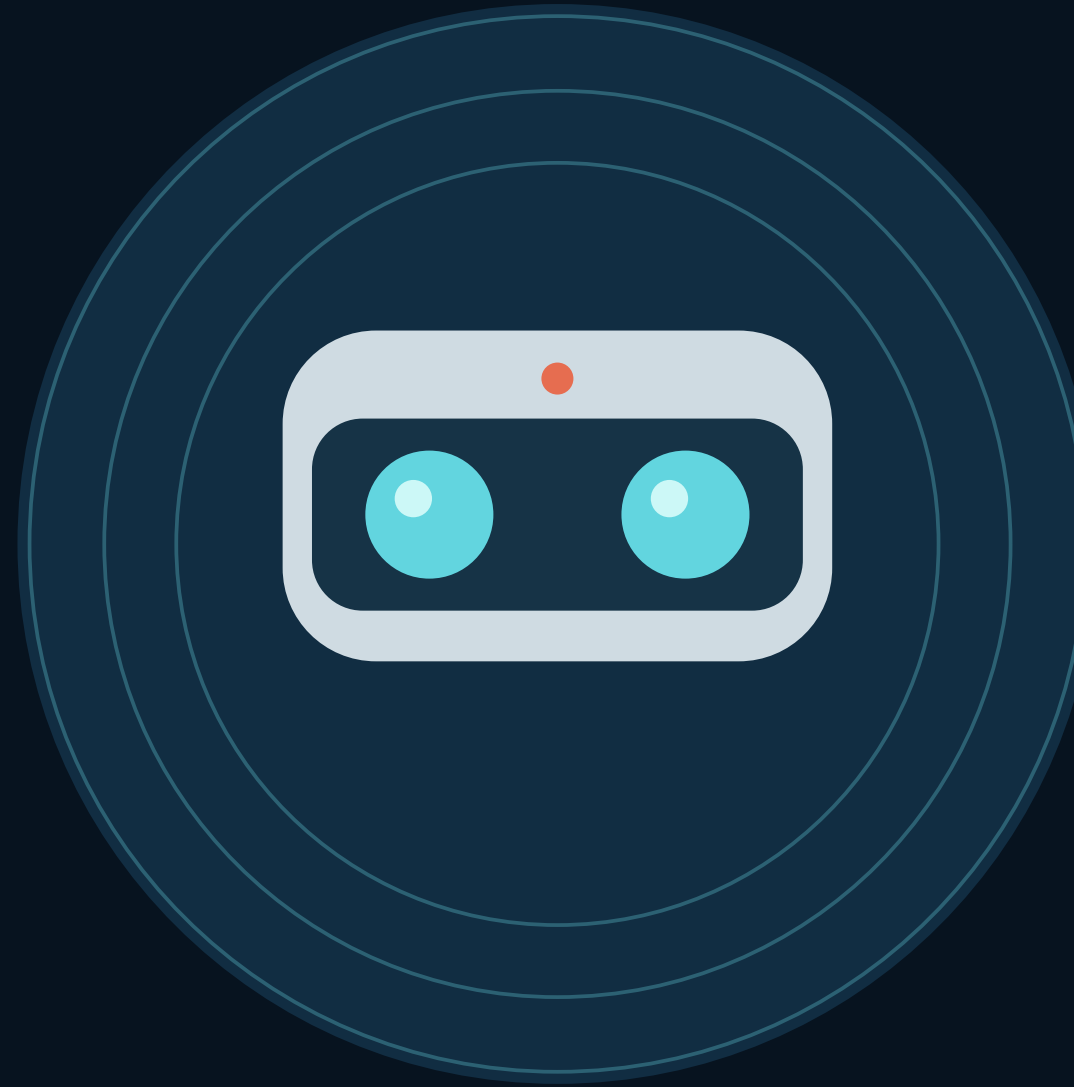


R.O.B. VISION

OPTICAL ROBOT COMPANION / MK.01



TECHNICAL REFERENCE

Architecture, optics, networking, hardware, and verification

DESIGN PREVIEW 0.1 / 24 SEPTEMBER 2026

SIMULATION AVAILABLE · PHYSICAL BUILD NOT YET VERIFIED

CHAPTER 01 / DESIGN PREVIEW

R.O.B. Vision technical architecture

Status: local controller prototype plus browser simulation. The UNO Q is the planned deployment target; R.O.B. and all accessories are virtual. The Python service now implements ROM-pattern decoding, optional OpenCV capture, virtual accessory state, and an HTTP dashboard connection. The camera and UNO Q have not been bench validated. Gyromite's LAN Controller 2 return path remains unimplemented. See the virtual system contract.

Components and authority

Component	Owns	Does not own
Game display/emulator	Game rendering and optical flashes	R.O.B. Vision's virtual pose
Optional RetroPie launch hook	Exact configured game ID and launch/exit state	Motion commands or ROM-content proof
UNO Q camera decoder	Timestamped frames, flash region, complete-command validation	Virtual pad assertions from partial flashes
UNO Q mode adapter and virtual robot controller	One accepted action, bounded pose, gyro/disc state, virtual pad state, ordered events	Professor location or unobserved game outcome
UNO Q LAN sender	Fresh red/blue virtual Controller 2 states	Arbitrary emulator commands
Paired game-host receiver	Virtual Controller 2 button state and timeout release	Trusting stale or unpaired packets
Browser dashboard	Live R.O.B., pieces, controls, explanations, and history on laptop/iPad/phone	Independent game state or button authority

The currently available dashboard substitutes a local scripted state machine for the UNO Q stream. Its SIMULATION status must remain visible until a real UNO Q session connects.

Optical to virtual action

1. Gyromite or Stack-Up displays an optical command on the cabinet LCD/OLED.

2. The UNO Q timestamps camera frames, samples the calibrated flash region, and decodes one complete command. Ambiguous or partial messages are rejected.
3. The mode adapter interprets the primitive for the active game. Gyromite up/down spans two calibrated levels; Stack-Up up/down spans one, according to the [ROM analysis](#).
4. The virtual controller accepts one command when ready, applies a bounded pose or grip change, and updates virtual object locations where the action legitimately changes them.
5. It publishes a versioned snapshot and ordered events over LAN. Browser clients animate from those states.
6. In Gyromite, the virtual pad reducer sends any changed red/blue state through the paired LAN return path. The receiver exposes the configured Controller 2 button mapping to the emulator.

A launch hook can select the correct game adapter from exact configured ROM filenames, following the [game identification design](#). It does not bypass camera calibration or supply optical motion commands. Unknown games leave the controller idle.

Virtual Gyromite state

Track each gyro separately: `id`, `location`, `spin_phase`, `spin_started_at`, `spin_speed_or_estimate`, `held_by_robot`, and `upright`. Locations are `holder_a`, `holder_b`, `spinner`, `held`, `red_pad`, and `blue_pad`. The virtual spinner starts or restores spin; time and animation move a gyro through spinning, slowing, wobbling, and stopped phases. The browser demo uses a 55-second illustrative lifetime, not a measurement from original hardware.

The virtual red or blue pad is pressed when an upright spinning gyro occupies it, or when an unspun gyro is explicitly held down there by R.O.B. Moving, lifting, or tipping it releases the pad. Pad state is a deterministic output of the UNO Q's virtual model: there are no tray sensors. Both virtual buttons release on mode exit, game exit, session reset, sender timeout, or receiver timeout. A button transition and the corresponding visible state update must share one event sequence so the dashboard and game return path cannot silently disagree.

The [Gyromite booklet](#) says a single gate does not require a spinning gyro; holding an unspun one on its pad suffices. A spinning gyro frees R.O.B.'s hands to operate the other gate. A gyro no longer needed returns to its holder. Optical/player commands direct live play; the preview's scripted recovery is only a demonstration and must not be assumed to solve a level autonomously.

Virtual Stack-Up state

The browser stack model tracks five disc IDs, ordered contents of each of five trays, carried stack segment, station (1-5), height (six levels), and open/closed grippers. At the selected height, closing the grippers takes the contacted disc and all discs above it as one ordered segment; opening releases the full segment onto the target stack. A top-disc transfer is one special case. It validates height and station bounds, carried clearance, destination capacity, and conservation of all five discs before committing each command. Rejected actions leave state unchanged. This model currently drives the local scripted demo and manual buttons. The planned UNO Q service must own equivalent authoritative state. Memory mode requires a bounded plan for rapid commands; Bingo's simultaneous row-and-column no-command case must remain a no-op. Stack-Up's documented modes have no gyro-pad return path.

State and event contract

The UNO Q session has `BOOT`, `CALIBRATING`, `READY`, `BUSY`, `PAUSED`, `FAULT`, and `EXITED` states. Events carry session ID, monotonic sequence, timestamp, game/mode, source (optical, local, or hook), accepted/rejected command, virtual pose, object state, virtual pad state, and game-link freshness. A reconnecting browser receives a snapshot before subsequent events. The UI shows commanded action, modeled result, and connection status separately. It never labels an unobserved game result as confirmed.

The virtual controller accepts semantic actions with bounded coordinates and valid object transitions. A second optical command while busy is rejected with a visible event by default; a bounded queue is allowed only after mode-specific timing tests. A complete duplicate transmission is a new event, not a continuation of the previous flash.

Failure and reset behavior

Condition	Controller behavior
Partial/unreadable flash	Reject; no virtual action.
Game exit or unknown title	Stop game-specific actions; release both Gyromite buttons.
LAN sender/receiver timeout	Release both virtual buttons; mark return path unavailable.
Browser disconnect	Preserve UNO Q state; reconnect from snapshot.
Impossible virtual transfer	Reject with diagnostic; do not mutate piece stacks or buttons.
Local demo Home/Stop	Cancel pending script and spin/recovery timers; release local virtual buttons.

The dashboard's **Emergency Stop** is currently a local simulation cancel control. In this virtual design it is not a physical motor–power cutoff. The integration must protect optical input and paired LAN messages against stale, malformed, or unauthorized data; see [safety and security](#) and [network architecture](#).

Implementation map

Available: the standalone browser simulation; `controller/optical.py` for ROM-derived patterns; `controller/model.py` for virtual game state; `controller/service.py` for optional OpenCV capture and HTTP snapshots; `dashboard/live.js` for the browser connection; exact-name resolution and optional launch notification in `tools/`; and ROM analysis. Planned: measured UNO Q camera settings, ordered live event stream, paired Gyromite Controller 2 receiver, installer, and replay diagnostics. No pin map or motor firmware is required for the virtual R.O.B.

CHAPTER 02 / DESIGN PREVIEW

Original R.O.B. manual: historical notes

Source: [R.O.B. instruction manual transcript](#) at Atari HQ. This is a paraphrased reading of the original physical toy, not a hardware specification for R.O.B. Vision.

The original robot grasped/released, raised/lowered, and rotated/carried objects. Its operator aimed its eyes at a CRT test signal. A head light indicated optical reception and readiness, went out while it moved, and returned when ready for another command. R.O.B. Vision represents those movements and states in a browser; its UNO Q camera is separately mounted and watches a modern LCD/OLED. There is no physical R.O.B. head, arm, or accessory.

The manual warns about glare, fluorescent light, blocked sight lines, overlays, and overly bright images. Those remain useful camera–test cases. The modern UI must separate **flash visible** from **complete command decoded**, and show TEST → READY → BUSY → READY clearly. The original approximate CRT distance is only historical context; actual camera framing and exposure need calibration on the chosen display.

The original manual's battery, mechanical, and spinning–gyro safety instructions apply to Nintendo's toy. This virtual project instead needs robust command validation, correct virtual piece state, a fresh paired Gyromite return link, and clear simulation/live labels. See [optical input](#) and the [virtual system contract](#).

CHAPTER 03 / DESIGN PREVIEW

Gyromite manual: design and test notes

Source: [Gyromite instruction booklet transcript](#) at Digital Press. This is a paraphrased summary of Nintendo's physical game accessory, translated into rules for a virtual R.O.B. The transcript includes editorial notes and possible transcription mistakes; confirm game behavior with a user-supplied game before claiming compatibility.

Original accessory and virtual equivalent

The original set has one motorized spinner, two gyro holders, red/blue gyro trays, removable hands, and a holder for Controller 2. A gyro on the blue tray lowers the blue gate; one on red lowers the red gate. The original trays press Controller 2 mechanically. R.O.B. Vision draws these objects and derives a **virtual** red/blue button state in UNO Q software, then sends it over Wi-Fi or Ethernet to a paired virtual Controller 2 receiver. There are no physical gyros, trays, or contact sensors in this project. The transcript does not unambiguously map tray colors to Controller 2 A/B; test this in the running game.

The booklet says that **one gate can be operated without spinning a gyro**. R.O.B. may hold an unspun gyro down on a pad, then lift it to release. When both gates need control, a spinning gyro remains upright on one pad while R.O.B.'s hands move the other. The original setup begins with both gyros in holders, and the instructions describe R.O.B. moving a gyro between holder, spinner, and tray. The virtual controller therefore treats holders as resting positions and the spinner as an optional step driven by game needs.

Game modes

Mode	Manual behavior	Virtual design implication
Test	Optical aiming signal	Provide camera framing and decoder diagnostics before play.
Direct	Up/down/left/right/open/close go straight to R.O.B.	Validate six optical commands and show each virtual movement.
Game A	Player controls the professor on a dark screen; Start enters a blue robot-transmission screen	Decode complete robot commands in the correct mode; avoid claiming to know professor position from flashes.
Game B	Professor walks while Controller 1 commands R.O.B. directly	Show ready/busy state and virtual-pad-to-game latency.

Acceptance questions

- Capture Test and six Direct commands on the intended LCD/OLED and emulator.
- Demonstrate held unspun press/release of one virtual pad.
- Demonstrate two independent gyro positions and spin-down phases, red/blue button state, and return to holders.
- Measure red/blue-to-A/B mapping, receiver freshness, and actual game gate response. A local visual button state is not proof that the emulator accepted input.
- Test Game A's transmission switch and Game B's direct command cadence.

Original booklet warnings about touching real spinning gyros are historical context. The [virtual system contract](#) is the current product scope.

CHAPTER 04 / DESIGN PREVIEW

Stack-Up manual: design and test notes

Source: [Stack-Up instruction booklet scan](#) at Digital Press. This paraphrases the original accessory and game rules. R.O.B. Vision models the robot and pieces virtually; no physical tray or disc is built.

Virtual set and starting layouts

The booklet shows five colored ring-shaped pieces—red, white, blue, yellow, and green—on five numbered trays around R.O.B. Tray 3 is directly in front. Direct and Memory begin with all five on Tray 3, top to bottom red, white, blue, yellow, green. One-player Bingo also begins with five on Tray 3, in any color order. Two-player Bingo begins with three on Tray 3 and one each on Trays 2 and 4. These layouts become UNO Q virtual-state profiles, with matching browser graphics. At the start of a virtual session, the pose is centered over Tray 3, at the highest level, with hands open.

Command and stack behavior

The game sends one of six movement commands—left, right, up, down, open, close—per completed signal. The original robot has five tray stations and six height levels; Stack-Up up/down moves one level, unlike Gyromite's two-level movement. Closing at a lower disc can lift that disc and every disc stacked above it together. The virtual controller therefore carries an ordered **stack segment**, not just one disc. Opening places that segment, preserving color order; all five discs remain accounted for. The game does not send a disc ID, destination tray, target pattern, or confirmation that the player's arrangement is correct. Those are inferred from our virtual pose and stack state or seen by the player on the game screen.

The current scripted preview applies one virtual command per step. It first moves red alone from Tray 3 to Tray 4, then closes around blue at level 3 and carries blue plus white to Tray 2. The discs keep their bottom-to-top order, and all five remain accounted for. Local command buttons use the same virtual stack model. This is not evidence that the camera decoded any command; live play must animate each received primitive independently, including repeated commands and Memory playback.

Modes

Mode	Booklet behavior	Virtual design implication
Test	Optical aiming signal	Reuse camera calibration and readiness check.
Direct	Professor lands on six command keys	Decode and display left, right, up, down, open, or close. Player scores/advances with Start after arrangement.
Memory	Up to 100 programmed commands at selected speed, then END	Measure cadence; bound queued virtual actions and report misses. END is a programming marker, not a robot movement flash.
Bingo, one player	Row or column completion sends a command	Apply only a complete validated optical command.
Bingo, two players	Two human controllers and a different starting layout	Validate separately; keep human controller input on the cabinet.

The booklet says a simultaneous completed row and column is a no-command condition until one line changes. Do not turn that pattern into two virtual actions. The flash stream does not report a target pattern or a victory result. The UNO Q knows its own modeled disc stacks but cannot infer the game's scoring outcome without another source. In Direct, Memory, and one-player Bingo, the human presses Start to request scoring after matching the displayed arrangement; a successful animation alone does not advance a round.

Stack-Up's documented modes have no Gyromite-style accessory that presses Controller 2. Its tray model needs no button return path to the game. The browser preview now demonstrates both a one-disc transfer and an ordered two-disc carry. Future optical play must conserve all five discs, update one modeled placement per valid open

command, and handle Memory timing without silent command loss.

See the [virtual system contract](#), [ROM signal analysis](#), and [verification plan](#).

CHAPTER 05 / DESIGN PREVIEW

Game identification from RetroPie launch events

Status: local resolver prototype; no cabinet hook, network sender, or robot receiver is installed. The game still flashes commands for the head camera to decode. **Launch identification supplies context and the correct accessory guide; it does not authorize motion or replace the optical input.**

Decision

Use the same approach as VirtualGlove: append a small call to RetroPie's existing `runcommand-onstart.sh` and `runcommand-onend.sh` chain. At launch, RetroPie supplies system, emulator, ROM path, and command. Match the **exact ROM basename including** `.nes`, `.zip`, or `.7z`, case-insensitively, against an explicit registry. At exit, clear the active game. Preserve the cabinet's existing VirtualGlove, controller, RGB, and trackball hooks; do not replace their scripts.

Only `nes` and `famicom` are eligible. An unregistered filename or other system resolves to idle, not a guessed game. A changed archive filename needs an explicit registry entry. This avoids recognizing an unrelated game merely because its name contains "Gyromite" or "Stack-Up."

The checked-in [registry](#) covers the user's Gyromite (World) and Stack-Up (World) `.7z` and `.zip` files plus their named `.nes` members. A read-only inspection of the supplied archives on 24 September 2026 found one 40,976-byte `.nes` member per archive. For each game, its `.zip` and `.7z` members had identical bytes. No ROM bytes, extracted files, or archive copies were added to R.O.B. Vision.

Current local check

The [resolver](#) accepts the same four launch arguments used by RetroPie and prints a small JSON event. It does **not** send that event over the LAN or edit a `runcommand` hook.

```
python3 tools/identify_game.py start nes lr-fceumm "/roms/nes/Gyromite (World).zip" ""
python3 tools/identify_game.py start nes lr-fceumm "/roms/nes/Stack-Up (World).7z" ""
python3 tools/identify_game.py end
```

The first two calls identify `gyromite` and `stack_up`; the last returns `idle`. The ROM argument is the **launched filename** reported by RetroPie, not the inner `.nes` member of an archive unless RetroPie actually launches that extracted file.

Planned cabinet-to-robot contract

```
RetroPie runcommand start/end
-> exact local registry match
-> authenticated, ordered LAN game-state event
-> UNO Q acknowledges and displays game/accessory context
-> optical test + virtual accessory profile checks
-> READY for validated screen-flash commands
```

The sender should include a session ID, monotonic sequence, system, sanitized ROM basename, game ID or `idle`, and launch/exit event type. It should retry for an authenticated acknowledgement without blocking the game launch. The robot should ignore duplicate or stale events, clear game readiness on an exit or unknown title, and clear stale game context after a bounded heartbeat/lease timeout. A queued acknowledgement is not proof that the robot is calibrated or ready.

The virtual robot should display `GYROMITE`, `STACK-UP`, or `UNKNOWN GAME` along with the source of identification. It should never animate a game command merely because a launch event arrived. An authenticated hook event selects the expected game profile, while the camera independently validates actual optical commands. If the two disagree, show a mismatch and refuse game-driven virtual actions until the operator resolves it. Manual game selection may support a non-RetroPie source, but must be explicit and pass the same optical and virtual-state checks.

Gyromite's paired tray-to-Controller-2 return path remains separate from this launch notification. Stack-Up needs no gyro-style button return in its documented modes. Both games may use the Wi-Fi/Ethernet launch event for context while receiving their movement commands through the camera.

Acceptance checks before installation

1. Simulate all six registered archive/member basenames, an unrelated NES ROM, and another system; verify only intended games match.
2. Launch both archive formats through the real RetroPie UI and record the **actual** system and ROM argument; adjust the registry only from that observation.
3. Verify start, exit, unknown game, back-to-back game launches, missed acknowledgement, and sender/robot restart. No stale game may stay armed.
4. Confirm all existing cabinet runcommand actions still execute in their original order and that hook failure never prevents the game from starting.
5. Compare the launch identity with camera-observed Test/Direct flashes. Only matching, calibrated signals may enable future game-specific virtual commands.

See [network architecture](#), [optical input](#), [configuration](#), and [verification plan](#).

CHAPTER 06 / DESIGN PREVIEW

Optical game input: camera watching a modern display

Primary approach

Mount a small adjustable camera with the UNO Q and aim it at the cabinet's LCD/OLED game display. R.O.B.'s eyes are drawn in the UI; there is no physical robot head. The camera watches the brightness changes when Gyromite or Stack-Up sends a command. UNO Q Linux converts the changes into one of the game's single-step virtual robot primitives. The camera stream supplies a small alignment preview in browser diagnostics. The exact ROM-derived messages and actions are in [ROM signal analysis](#).

The original Nintendo design used a photo detector looking at a CRT. Its patent describes white/black image changes, a start/trigger structure, and updates aligned to vertical blanking. This supports the basic idea of optical decoding, but **does not prove that any particular camera and modern display combination will capture the pulses correctly**. The modern screen may show the flashes while its refresh behavior, scaling, image processing, pixel persistence, or brightness controls alter their timing. The camera can also miss or smear transitions through its frame rate, exposure, rolling shutter, or automatic gain.

Processing pipeline

```
camera frames + timestamps
-> locate game display / flashing region
-> lock a small region of interest
-> measure normalized brightness per frame
-> recover 13 dark/green bits at the game-frame clock
-> validate `000101w1x1y1z` and the selected game's command set
-> assign one event to one complete transmission
-> send one bounded primitive to the planner
```

Start with fixed camera placement and a user-adjustable rectangle on the browser dashboard. Automatic flash-region detection can come later. Bench-test delivered 120/240 fps camera modes, short fixed exposure, fixed gain, and fixed white balance; the roughly 60 Hz game bit rate cannot be trusted from nominal camera frame rate alone. Record actual frame timestamps. The decoder should expose raw brightness, threshold, bit timing, candidate bits, final command, confidence/rejection reason, and last-seen time so failures are understandable. A second region away from the game screen helps reject ambient flashes.

The ROMs show that Gyromite sends **two-level** up/down commands while Stack-Up sends **one-level** up/down commands. Both send one-station left/right and open/close. The ready-light command and alternating alignment-test flashes cause no arm movement. Repeated identical separate transmissions are valid game commands, especially

during Stack-Up Memory playback; duplicate suppression applies only to overlapping detections of the same captured frames.

An accepted optical command is **not** a raw browser animation request. It goes through mode, virtual pose, object-state, and busy checks on the UNO Q. The system must show NO SIGNAL, UNREADABLE SIGNAL, or COMMAND ACCEPTED distinctly. A malformed or incomplete pulse train produces no virtual action.

The [original instruction manual transcript](#) distinguishes a flickering test indication, a steady ready indication, and an unlit indicator during movement. It also notes that an overly bright screen can pass the test indication yet fail movement commands. For R.O.B. Vision, **flash visible**, **command decoded**, **ready to act**, and **busy moving** must be separate states backed by actual evidence. The historical 1–2 m CRT placement is a test candidate, not a camera specification; check the usable distance experimentally on the chosen LCD/OLED.

Bench feasibility test before camera selection

1. Use the intended LCD/OLED, emulator/core, refresh setting, video filters, and actual game. Aim the candidate camera at the game's flash area from the planned robot position.
2. Record lossless or minimally compressed frames with monotonic timestamps. Capture the game's own test flash and every movement command many times, alongside idle gameplay. Include Gyromite's two-level movements and Stack-Up's one-level movements.
3. Plot the brightness trace and compare it with a fast reference light sensor if needed. Determine whether each pulse and gap remains distinguishable after display and camera sampling.
4. Verify complete 13-bit command identification against the [ROM-derived patterns](#) across reasonable brightness, ambient light, viewing angle, and scaling settings. Check false triggers during ordinary gameplay and while R.O.B.'s own eye/display lights are active.

Include glare, fluorescent lighting, volume/menu overlays, occlusion, exposure saturation, and a small shift of the camera or display. The original manual identifies these as reception risks; test their modern equivalents instead of assuming the camera behaves like its photosensor.

5. Aim for **at least 99 correct decodes in 100 repeated transmissions per command, with zero idle false triggers in a representative play session** before committing the camera to the mechanical design. This is a project acceptance target, not a claim of current performance.
6. Repeat after any camera, panel, video-mode, or emulator change. Keep sample recordings and decoder traces as reproducible test fixtures; do not put ROMs or copyrighted game footage in the public repository.

For Stack-Up, the [booklet](#) adds two specific captures: Memory playback at each offered speed, and Bingo when one key completes a row and column simultaneously. The latter is a documented no-command condition despite simultaneous flashing. The decoder and planner must make no move. A camera exposure that merges two flashes cannot be accepted as a valid single command.

If the candidate IMX219 camera cannot sample the relevant transitions reliably, evaluate a faster camera or a small reference photodiode beside it. The camera can still provide framing and the visual preview. The user-preferred camera path remains the first experiment; an emulator hook is a later fallback. Choose the final capture device from traces, not advertised resolution.

Build implications

- Reserve one eye for a clear, stable camera aperture with adjustable focus/aim and a removable lens window. Keep decorative eye LEDs out of its optical path.
- The UNO Media Carrier is a candidate because Arduino documents MIPI-CSI camera inputs compatible with IMX219 modules. Physical cable routing must accommodate head rotation without excessive bending.
- The browser dashboard on laptop, iPad, or phone should make the virtual robot and accessories primary, with command, mission, and modeled pose visible. Camera alignment and flash traces belong in a diagnostic panel.
- The cabinet-to-robot network is used for sensed controller feedback and optional hooks; seeing the game flashes requires no incoming network command.

References

- [Nintendo photosensing video game control patent](#) – original brightness-code mechanism and timing tied to CRT frames.
- [Arduino UNO Media Carrier](#) – supported internal camera and display connector paths.
- [Original R.O.B. instruction manual transcript](#) – historical setup, ready/busy indication, and optical troubleshooting; see our [design notes](#).

CHAPTER 07 / DESIGN PREVIEW

Gyromite and Stack-Up ROM signal analysis

Status: Static analysis of the user-supplied Gyromite (World) and Stack-Up (World) ROMs, 24 September 2026. The ZIP and 7z copies of each game contain identical NES data. No ROM or extracted graphics is stored in this project. The derived patterns have not yet been captured from the user's LCD/OLED and camera.

Run `python3 tools/analyze_rob_roms.py --gyromite '/path/to/Gyromite (World).zip' --stack-up '/path/to/Stack-Up (World).zip'` to reproduce the command report from the archives in place. The program rejects unexpected ROM layout or changed evidence bytes. The inspected full-file SHA-256 hashes are `bf1b323ba39c84b964f93127598b267ff3f16cbf99c533ae1e9b8332232b3e5b` (Gyromite) and `3ab6bc99246783a6bf7083481027fe358ccdb147dcf1165eaf08aaf4e1b06548` (Stack-Up). Each is an iNES mapper-0 image with 32 KiB program ROM and 8 KiB character ROM.

What the games transmit

The screen flashes represent **one robot primitive per command**, not a complete task such as “move the red gyro to its tray” or “solve this Stack-Up pattern.” A 13-frame message has the form:

```
000101w1x1y1z
```

0 is the dark frame (NES palette \$0F), 1 is the green frame (\$2A), and wxyz selects an action. The pattern is consistent with the independently documented [NESdev R.O.B. interface](#) and [original hardware protocol research](#). At a roughly 60 Hz NES video rate, the 13 data frames span about 217 ms; the final visible duration and gaps still depend on emulator and display behavior. Stack-Up's transmission routine schedules an additional dark frame after the 13 data frames.

Action	wxyz	Full 13-frame pattern	Gyromite	Stack-Up
Open grippers	1010	0001011101110	Yes	Yes
Close grippers	0110	0001010111110	Yes	Yes
Turn left one station	0100	0001010111010	Yes	Yes
Turn right one station	1000	0001011101010	Yes	Yes
Raise two levels	0101	0001010111011	Yes	No
Lower two levels	1101	0001011111011	Yes	No
Raise one level	1100	0001011111010	No gameplay table	Yes
Lower one level	0010	0001010101110	No gameplay table	Yes
Head ready light on	1001	0001011101011	ROM command table	Stack-Up test path

The eight-bit command byte is the message after the five leading bits: for example, open is \$EE, close \$BE, left \$BA, right \$EA, Gyromite up/down \$BB/\$FB, Stack-Up up/down \$FA/\$AE, and ready light \$EB. The ready-light message is a status operation. It must never move an axis.

ROM evidence

- **Gyromite:** The program at \$A679-\$A697 indexes a palette-choice table at \$A6FC, with 13 entries per command slot and the entries read in reverse order. Valid populated slots 1, 2, 3, 5, 6, 7, and 8 decode to the rows above. Slot 4 is deliberately skipped in the controller selection logic at \$A591-\$A5A2; its table area is not a valid optical command. Palette-selection values 3 and 4 resolve through the table at \$B35E to the all-black palette at \$B404 and all-green palette at \$B428 respectively. This ties the table bits to actual on-screen light, rather than just matching known codes by coincidence.
- **Stack-Up:** The routine at \$B1DE starts a transmission and \$B1E7-\$B237 builds successive frames. It uses the template at \$B24A: six fixed preamble bits followed by four variable bits separated by fixed green bits. The variable bits come least-significant-bit first from command values 1-6. Palette selections 2 and 3 point via \$82E8 to the black and green palettes at \$B598/\$B5AC. The ROM's final loop pass is an extra dark frame. A separate test path calls the sender with value 9 for the head light; that is not a movement command.
- **Cross-check:** The byte patterns and action names agree with the [NESdev hardware interface](#). The original [Nintendo photosensing patent](#) describes frame-aligned light code transmission, but this report's per-game command table comes from the supplied ROMs.

These addresses refer to the CPU view of these exact 32 KiB program images (\$8000-\$FFFF). The static analysis establishes the encoded message and palette choices. It does not establish the optical fidelity of a particular emulator, TV, or camera, or every mode-specific command cadence.

Camera decoder we should build

1. Aim and lock an adjustable region of interest at the game display. Use the game's **full-frame dark/green changes**, not sprite or text recognition. Keep a background/control region to detect room-light changes and glare.
2. Capture timestamped luminance faster than the roughly 60 Hz bit rate. Start bench trials at 120 or 240 captured frames per second with fixed exposure, gain, and white balance; verify the delivered timestamps and use a short enough exposure to separate adjacent game frames. These are test settings, not a guarantee that a chosen camera can achieve them at the required resolution.
3. Turn samples into a frame-clocked dark/bright sequence. Search for the complete 000101 preamble, then four variable bits at the specified positions with the three required intervening green bits. Require plausible bit widths and a complete message; reject missing, merged, or ambiguous samples.
4. Accept only the action set for the identified game. A Gyromite two-level move and a Stack-Up one-level move are distinct actions. Keep the ready-light and test flashes outside the motion path. Ignore arbitrary game brightness changes that fail the full pattern.
5. Assign one event ID to one complete optical transmission. Repeated **separate** identical commands are legitimate, especially in Stack-Up Memory; do not suppress them by command value or by a broad time window. Suppress only extra detections of the same captured frame interval.
6. Record the sampled brightness trace, bit timing, candidate pattern, accepted/rejected result, source game, and action disposition. The browser dashboard should distinguish FLASH SEEN, CODE DECODED, ACTION ACCEPTED, BUSY, and FAULT.

Game identification can follow the VirtualGlove-style exact launch-filename registry already described in [Game Identification](#). It selects the decoder's allowed action set and the matching virtual accessory profile; it is not evidence that a flash was decoded. A missing or unknown launch event keeps game-specific virtual actions disabled until the game is selected locally and verified.

What the virtual robot should do

Every accepted optical message advances **one bounded virtual primitive**. The UNO Q owns the virtual state and the browser animates it. No physical motion controller is part of this design.

Optical event	Robot operation	Display/state requirement
Left / right	Rotate one virtual station	Show station before/target, fixture occupancy, busy state.
Gyromite up / down	Move two virtual vertical levels	Show both intermediate and final levels; reject an out-of-range target.
Stack-Up up / down	Move one virtual vertical level	Show target level and held-disc state.
Open / close	Open or close the illustrated grippers once	Update a piece only when the modeled grasp or release is valid.
Ready-light / test	Update optical or visual readiness only	Never move virtual pieces.

For **Gyromite**, a sequence of these primitives transfers a virtual gyro between holder, spinner, red pad, and blue pad. The UNO Q derives virtual pad state from its object model and sends that state to the paired LAN Controller 2 receiver. The flash stream alone does not identify the professor's location or prove that the game accepted the returned button. The model tracks which gyro is held and which pad is pressed.

For **Stack-Up**, the same primitives manipulate five virtual colored discs across five numbered trays. The pose also includes six height levels; closing at a lower disc can carry it and every disc above it as an ordered segment. The flash stream has no disc ID, target-pattern, or victory message. The UNO Q can know its own modeled stack state, but not the game's desired pattern without another validated source. Memory mode can transmit successive commands faster than the animation finishes: measure cadence and use a bounded, visible queue only if tests show it is needed; otherwise report a missed-command state. In the Bingo simultaneous row-and-column case described in the manual, the game sends no command until one line changes; the camera should yield no valid movement message.

Next validation gate

Capture **all nine patterns above plus idle, test, Stack-Up Memory speeds, and the Bingo no-command case** on the intended LCD/OLED with the intended emulator and camera. Compare recovered bits against the ROM-derived rows; measure false triggers, missed bits, command spacing, and display processing delay. Until that capture succeeds, the ROM work specifies the decoder but does not prove the proposed camera will read the real screen.

CHAPTER 08 / DESIGN PREVIEW

Laptop, tablet, and phone dashboard

Status: design contract plus local simulation. R.O.B. and all accessories are virtual. The current browser preview does not connect to the UNO Q camera service or game host.

Visual hierarchy

The main pleasure of this interface is watching virtual R.O.B. act on visible game pieces. Robot motion, the held object, and the accessory station should read as one continuous event. The game screen and camera image support that event; the robot and pieces remain the main view. The live implementation should animate accepted optical commands from the UNO Q's virtual state and keep piece occupancy consistent with the same state used for Gyromite button output. Cosmetic eye and gyro effects may add personality, but must stay distinct from validated game input.

1. **R.O.B. model is the main view.** Keep the friendly face while drawing two outward-bent hand tips that converge around one object. Each hand is attached to one arm; neither hand contains its own little pincer. The shoulder hubs remain visibly connected to a shared carriage while the hands rise/lower and sweep left/right. Turn the head in the same direction as the sweep. Place the hands visually in front of the prop so the contact is readable. Animate body station, vertical level, shared grip state, and current/next action. Show when a pose comes from a scripted preview or a validated optical command.
2. **Game table is always visible in the selected mode.** Gyromite shows spinner, two gyro docks, red/blue button trays, and both gyros. Stack-Up shows five numbered trays and five colored discs, including the mode-specific starting stack. Show modeled object location, virtual pad state, and whether each update came from a scripted preview or a validated command.
3. **Status and activity remain legible.** Show game identity source, optical decode, accepted/rejected primitive, busy state, virtual pose, virtual pad state, and Gyromite receiver freshness. A decoded command, a virtual action, and a game-host acknowledgement are different events.
4. **Camera video is secondary and optional.** A small tile may show a downsampled view of the game display for orientation. It cannot prove an optical command. If unavailable, the tile says so and the rest of the dashboard still works.
5. **Detailed camera analysis belongs in diagnostics.** The UNO Q camera watches the cabinet display. Show its region of interest, brightness trace, and frame timing on demand. Keeping the detailed feed off the main screen leaves R.O.B. and the pieces as the focus.

Device layout

The UNO Q should serve the same browser application and state stream over the trusted LAN to a laptop, iPad, or phone. Desktop and wide tablet layouts can keep the R.O.B. model beside mission and telemetry cards. A phone stacks the model, game table, status, and controls vertically, with touch-sized controls and no horizontal scrolling. The observer tile never displaces the model or hides a fault. Reconnection should request a fresh state snapshot and show data age; it must not replay movement commands.

The current static preview has been checked in a browser at 390 px and 820 px widths. Device touch behavior, real LAN latency, and observer-video bandwidth remain untested.

Proposed live event contract

The UNO Q, not the browser, owns the game session and virtual robot state. Publish a versioned snapshot on connection, then ordered events with session ID, monotonic sequence, timestamp, and freshness. Include game/mode, optical decode and rejection reason, accepted primitive, virtual pose/grip, gyro or disc locations, virtual pad states, RetroPie receiver status, and faults. Browser clients render this stream and can reconnect without changing game state. Optional operator controls send bounded intents through the same virtual controller.

If observer video is enabled, deliver it separately at an adaptive low rate. The command decoder must consume the head camera's original timestamped frames locally on the UNO Q rather than a browser video stream. The user should be able to hide either preview without disrupting optical decoding or motion.

Current preview boundary

The finite Gyromite demo first shows R.O.B. holding an unspun gyro on one virtual pad, then runs a two-gyro spin relay. Each gyro has an illustrative 55-second spin-down clock, tips, and releases its virtual pad. R.O.B. re-spins each once, waits for both to stop again, returns them to their holders, and ends in a stable COMPLETE state. Both speed bars and red/blue virtual button states remain visible. HOME, mode change, demo restart, and simulation stop cancel pending actions. The UNO Q will own these virtual states during connected play; no physical tray contact is involved.

The [dashboard](#) animates R.O.B. and his virtual accessories together. Gyromite shows two matching gyros, two holders, the right-side spinner, and the red/blue button pads. Its finite demo ends with both gyros stored. Stack-Up shows five trays and all five discs. Its command-by-command demo moves red from Tray 3 to Tray 4, then grips blue and white together and places both on Tray 2. The Live Model, Accessory Bay, and Game Table all read the same virtual stack state. Local Stack-Up controls apply the same one-level and one-station actions and reject blocked moves without changing the discs. The art uses a three-quarter perspective, not measured hardware trajectories. Camera-driven play is still pending.

Pose Lab is a character-only motion sandbox. It shows no gyro, disc, or game fixture. Its short greeting and the manual Pose Preview controls let the viewer inspect turns, arm height, and the shared hand grip without claiming an optical command. The sidebar uses a R.O.B. portrait in this mode, and the game-table area explains that there are no

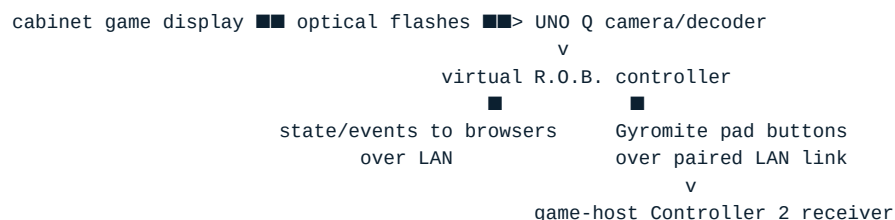
game pieces. The [ROM signal table](#) and [technical architecture](#) specify how connected play will drive visual state.

The original [Gyromite manual](#) describes opening and closing the arms, raising/lowering, and turning/carrying. The user-provided [R.O.B. photo reference](#) and [physical demonstration](#) inform the silhouette and station presentation. The visual design keeps the current expressive face; its station paths are virtual art choices, not measured hardware trajectories.

CHAPTER 09 / DESIGN PREVIEW

R.O.B. Vision network architecture

R.O.B. is a virtual model owned by the Arduino UNO Q. The cabinet displays the game; the UNO Q camera watches the optical flashes. A laptop, iPad, or phone receives the UNO Q's state and renders R.O.B. and every accessory. There is no physical robot or tray sensor and no USB data tether to the cabinet.



The optional RetroPie start/end hook sends exact configured game identity to the UNO Q. It selects a mode but is not a movement command. The optical decoder supplies movement commands. If camera capture proves infeasible, a future emulator adapter may supply equivalent validated commands through an authenticated channel; that fallback is not implemented.

Gyromite return path

The UNO Q derives red/blue virtual pad states from its virtual gyro and hand model. A spinning upright gyro on a pad presses it; an unspun gyro explicitly held down by R.O.B. also presses it. Moving or tipping it releases it. These short-lived states are sent to a paired receiver that exposes a virtual second controller to the emulator. The original booklet does not establish the red/blue-to-A/B mapping clearly enough to hard-code it; measure the mapping in a running game.

Packets need a session identifier, monotonic sequence, version, authentication, both current button states, and a bounded freshness period. The receiver releases both buttons on timeout, unpairing, game exit, or invalid input. A local socket send is not evidence that the emulator accepted a button; receiver status belongs in the dashboard.

Network reconnection does not replay an old held state without a new fresh snapshot.

Stack-Up's documented modes do not use this Gyromite tray-button return path. Its virtual piece state still streams to browser clients.

Browser stream and privacy

Browsers subscribe to a versioned snapshot followed by ordered events. The UNO Q remains authoritative if a tab closes or reloads. The main view shows R.O.B., virtual pieces, commands, and controller-link freshness. The game-screen camera image is used for calibration diagnostics, while the main view remains the robot. Limit any camera stream to paired/trusted clients and avoid recording ROM images by default.

Wi-Fi is the intended baseline. A compatible Ethernet adapter is optional; verify UNO Q power and interface support on the actual board. The application protocol should behave the same over either link. The local dashboard presently has no network service or game receiver.

CHAPTER 10 / DESIGN PREVIEW

R.O.B. Vision UNO Q hardware setup

Scope: build the small camera and network controller, not a physical R.O.B. R.O.B., hands, gyros, spinner, pads, and Stack-Up trays are virtual graphics and controller state. No motors, mechanical fixtures, tray sensors, motor drivers, or actuator power supply are required.

Planned controller assembly

- Arduino UNO Q with a supported supply and enclosure that allows ventilation and service access.
- Compatible camera and mount aimed at the cabinet's LCD/OLED flash area. Confirm the camera interface, frame mode, focus, exposure control, and cable length on the actual UNO Q before ordering a fixed enclosure.
- Built-in Wi-Fi as the baseline LAN path, or a compatible Ethernet adapter if wired service is preferred. The cabinet and UNO Q share a trusted local network; no USB tether to the cabinet is planned.
- A laptop, iPad, or phone opens the dashboard from the UNO Q over LAN. It is the principal R.O.B. display. An optional observer camera could show the room or controller, but it is not required for the virtual robot.

Commissioning order

- 1. Power the UNO Q and confirm the intended operating system, camera, and network interfaces independently.
 - 2. Aim the camera at the game display. Fix focus and exposure, then record timestamped bright/dark traces while using the game's Test mode.
 - 3. Validate optical framing and command decoding against both supplied ROM analyses before connecting any game return path.
 - 4. Serve the dashboard from the UNO Q and verify the same virtual snapshot appears on laptop, iPad, and phone.
 - 5. Pair the Gyromite LAN sender with the game-host receiver. Confirm red/blue virtual pad transitions, the actual A/B mapping, and release on disconnect or timeout.
 - 6. Run full Gyromite and Stack-Up sessions and retain logs for failures. Do not treat the current browser demo as evidence that this integration is complete.
- Cable routing and enclosure design should avoid blocking the camera view, heat vents, or wireless antennas. There is no moving R.O.B. mechanism to calibrate or guard.

CHAPTER 11 / DESIGN PREVIEW

R.O.B. Vision parts plan

The build is an electronic controller with a virtual R.O.B. display. The historical gyros, spinner, pads, trays, arms, and discs are **software models**, not parts to purchase or fabricate.

Item	Purpose	Status
Arduino UNO Q	Run camera capture, optical decoding, virtual robot state, dashboard service, and Gyromite LAN sender	Candidate controller; runtime integration unimplemented.
Compatible camera and mount	Watch the game's optical flash area on LCD/OLED	Interface, exposure, focus, and frame timing require bench proof.
UNO Q power supply and ventilated enclosure	Stable controller operation	Select after confirming board and camera current requirements.
Wi-Fi or compatible Ethernet adapter	Link UNO Q, dashboard devices, and game host	Wi-Fi baseline; Ethernet optional.
Laptop, iPad, or phone	Main animated R.O.B. display	Existing browser preview available.

Item	Purpose	Status
Game host with virtual Controller 2 receiver	Accept Gyromite red/blue button states	Proposed software; no live receiver yet.
Optional observer camera	Contextual image separate from the game-screen camera	Not required.

No servo, motor driver, home switch, gripper, physical gyro, tray sensor, spinner motor, or actuator supply is part of the current scope. Validate exact UNO Q camera and network compatibility before finalizing purchases.

CHAPTER 12 / DESIGN PREVIEW

R.O.B. Vision configuration reference

Status: prototype settings plus proposed hardware calibration. The static preview uses browser memory; the live Python controller serves shared state. Camera ROI, requested frame rate, and light threshold are initial code settings. No UNO Q settings service, pairing store, or Gyromite game receiver exists yet.

Area	Planned settings	Validation
Optical capture	Camera identity, exposure, gain, frame mode, flash region, timing thresholds	Stable timestamps and complete Test-mode command on the actual display.
Game identity	Exact configured ROM basenames and game IDs	No substring guessing; unknown title and exit clear context.
Virtual robot	Pose bounds, action duration, hand/grip rules, object locations	No impossible transfer or overlapping actions.
Gyromite	Gyro spin lifetime, wobble threshold, initial holders, pad rules, red/blue Controller 2 mapping	One held unspun press, two-gyro relay, independent release, measured A/B game response.
Stack-Up	Tray indices, starting disc order, six height levels, mode timing	Ordered disc groups move together when gripped below the top; no command loss in Memory mode.
LAN	UNO Q and game-host identities, receiver address, pairing key, packet interval, timeout	Fresh authenticated state only; both buttons released on timeout/exit.

Area	Planned settings	Validation
Dashboard	Text scale, reduced motion, camera diagnostic visibility	Touch and keyboard use on laptop, iPad, and phone.

Optical calibration records should include display and emulator mode, camera/mount, exposure, frame timing trace, ROI, date, and pass/fail result. Changing the display, camera mode, or video filter invalidates that calibration. Virtual gyro timing is a gameplay setting and must be identified as simulated; it is not a physical sensor threshold.

The only available local commands today are the browser preview and filename resolver:

```
python3 -m http.server 8000
python3 tools/identify_game.py start nes lr-fceumm "/roms/nes/Gyromite (World).zip" ""
python3 tools/identify_game.py end
```

The local camera trial uses OpenCV camera index 0, requests 120 fps, and samples the central 60% of the frame. Those are defaults for testing, not validated UNO Q settings. The service reports delivered fps and camera errors. There are no working game receiver or pairing commands yet.

CHAPTER 13 / DESIGN PREVIEW

R.O.B. Vision safety and security

R.O.B. and its accessories are virtual. This project has no robot arm, powered gripper, physical spinner, tray sensor, or actuator supply. The dashboard **Emergency Stop** cancels a local animation; it does not cut physical power. The main risks are incorrect game input, stale network state, camera privacy, and misleading UI status.

- Reject incomplete or ambiguous optical commands. A flicker or camera dropout must not repeat the last action.
- Bound every virtual pose and object transfer. A disc cannot appear on two trays, and one gyro cannot occupy two stations.
- The UNO Q, not a browser tab, owns virtual state and Gyromite pad output in the planned connected system.
- Pair the Gyromite sender and receiver on a trusted LAN. Version and authenticate messages, reject stale sequences, and release both Controller 2 buttons on timeout, exit, reset, or unpairing.
- Show simulation, optical validity, and game-link state separately. A drawn gyro on a pad is not proof that the emulator received its button state.

- Restrict camera diagnostics to trusted clients. Keep the game-screen camera aligned only to the intended display; do not record video by default. Do not package user ROMs in logs or releases.
- Browser clients reconnect from a snapshot. They do not resume a partially accepted action or silently assert old button states.

The original Gyromite booklet's warnings about touching a real spinning gyro describe Nintendo's physical toy. They are historical context, not a hardware hazard in this virtual implementation.

CHAPTER 14 / DESIGN PREVIEW

R.O.B. Vision verification plan

Current evidence: the browser preview demonstrates art, scripted virtual motion, independent gyro timers, virtual pad indicators, one recovery per gyro, and finite cleanup. It does not validate optical decoding, UNO Q hosting, LAN return input, or game behavior.

Gate	Test	Pass condition	Status
Gyromite preview	Run the unspun one-gate press, dual relay, spin-down recovery, and cleanup	Pad states stay independent; after one recovery per gyro, both gyros return upright to holders, buttons release, and COMPLETE remains stable	Browser preview observed.
Preview reset	Use Home and demo restart at different phases	Timers, queue, pieces, and buttons return to initial state	Browser implementation; verify after changes.
Stack-Up preview	Move red alone, then blue and white together; try a blocked release	Five discs conserved, colors stay ordered, and invalid moves leave state unchanged	Browser observed; five model tests pass.
Optical capture	Use actual game Test mode on intended LCD/OLED	Complete command decoded with timestamp evidence; idle produces no false command	Not run.
Game identity	Launch exact configured Gyromite/Stack-Up names and unknown titles	Correct context on start; idle on exit/unknown	Local resolver tested; live hook not installed.
Gyromite LAN output	Send red/blue virtual states and disconnect	Correct Controller 2 mapping; button releases on virtual lift, timeout, exit, and reset	Not run.

Gate	Test	Pass condition	Status
Multi-client UI	Open laptop, iPad, and phone simultaneously	Same UNO Q snapshot and ordered events; reconnection does not reset game	Not run.